Open Investment

Projeto Cidades ESG Inteligentes - Gerenciamento de Projetos Sociais

Documentação técnica - Práticas de DevOps

FIAP | Análise e Desenvolvimento de Sistemas | Turma 2TDSOC
Integrante: Lucas Gabriel Neres Sant'Ana - RM 565612

1. O projeto

API REST em Java 17 / Spring Boot 3.2 com Oracle XE 21c para gestão de projetos sociais no contexto ESG (pilares Social e Governance): controle financeiro com bloqueio de orçamento e alertas, impacto social, voluntários, beneficiários e trilha de auditoria. 32 endpoints em 8 controllers, autenticação JWT com perfis ADMIN/GESTOR/USER, Flyway para o schema e Swagger UI.

Componente	Tecnologia
Linguagem / Framework	Java 17, Spring Boot 3.2.5
Segurança	Spring Security 6 + JWT (JJWT 0.12)
Banco	Oracle XE 21c (container) + Flyway
Testes	JUnit 5, Mockito, AssertJ, H2 em memória
Build	Maven 3.9
Containerização	Docker (multi-stage) + Docker Compose
CI/CD	GitHub Actions

2. Pipeline CI/CD (GitHub Actions)

Arquivo: **.github/workflows/ci.yml**. Dispara em push e pull request nas branches main e develop. Os jobs de deploy só rodam em push: **develop = Staging** e **main = Produção**, cada um vinculado a um GitHub Environment.

Job	Quando	O que faz
build-and-test	Todo push/PR	Java 17, mvn package -DskipTests (build), mvn test (testes), publica relatórios surefire e o JAR
docker-build	Após testes OK	docker build da imagem (Dockerfile multi-stage) e listagem da imagem
deploy-staging	Push em develop	docker compose -p openinvest-stg --env-file env/staging.env up -d --build --wait; smoke test na porta 8081; resumo no job; derruba o ambiente
deploy-production	Push em main	Mesmo fluxo com -p openinvest-prod e env/production.env; smoke test na porta 8080

Smoke test (scripts/smoke-test.sh): (1) /actuator/health = UP; (2) login com o usuário do seed e recebimento do token JWT; (3) GET /api/projetos com token = 200; (4) o mesmo GET sem token = 401. Qualquer falha reprova o job.

Proteções: deploy depende de build, testes e imagem; DB_PASS e JWT_SECRET vêm de GitHub Secrets quando configurados (senão usam os valores de demonstração de env/*.env); staging e produção usam projetos Compose, redes, volumes e portas separados.

Limitação assumida: o deploy é efêmero. A stack real (Oracle + API) sobe no runner do GitHub, é validada e derrubada ao fim do job; não há servidor externo permanente. A evidência é o resumo do job e os logs do Actions.

3. Docker

Arquitetura: dois serviços (oracle-db e api) na rede app-net; volume oracle-data persiste o banco; configuração por variáveis de ambiente (API_PORT, DB_USER, DB_PASS, JWT_SECRET, SPRING_PROFILES_ACTIVE); a API só sobe quando o Oracle está healthy (depends_on com service_healthy). O mesmo docker-compose.yml atende os dois ambientes; muda apenas o arquivo env/*.env e o nome do projeto (-p).

	Staging	Produção
Projeto Compose	openinvest-stg	openinvest-prod
Porta da API	8081	8080
Usuário do banco	openinvest_stg	openinvest
Arquivo de variáveis	env/staging.env	env/production.env

Comandos usados:

```
docker build -t open-investment-api .
docker compose -p openinvest-stg  --env-file env/staging.env    up -d --build --wait
docker compose -p openinvest-prod --env-file env/production.env up -d --build --wait
bash scripts/smoke-test.sh 8081   # staging
bash scripts/smoke-test.sh 8080   # producao
docker compose -p openinvest-stg --env-file env/staging.env down -v
```

Imagem criada: open-investment-api (multi-stage: Maven + JDK 17 no build; eclipse-temurin:17-jre-alpine no runtime, usuário sem root, HEALTHCHECK).

Dockerfile:

```
# ============================================
# Dockerfile — Open Investment API
# Multi-stage build: compila com Maven e roda com JRE slim
# ============================================

# ---- Estágio 1: Build ----
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

# Copia apenas o pom.xml primeiro para aproveitar o cache do Maven
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copia o código-fonte e compila (sem rodar testes — Oracle não estará disponível)
COPY src ./src
RUN mvn clean package -DskipTests -B

# ---- Estágio 2: Runtime ----
FROM eclipse-temurin:17-jre-alpine

WORKDIR /app

# Cria usuário sem privilégios de root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

# Copia o JAR gerado no estágio de build
COPY --from=builder /app/target/open-investment-1.0.0.jar app.jar

# Porta padrão da aplicação
EXPOSE 8080

# Variáveis não sensíveis com padrão; DB_USER, DB_PASS e JWT_SECRET vêm do compose/.env
ENV DB_HOST=oracle-db \
    DB_PORT=1521 \
    DB_SERVICE=XEPDB1

# Health check para o Docker aguardar a app estar pronta
HEALTHCHECK --interval=30s --timeout=10s --start-period=90s --retries=5 \
```

```
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1

ENTRYPOINT ["java", "-jar", "app.jar"]
```

4. Prints do pipeline

Pipeline: jobs Build & Test e Docker Build

PRINT PENDENTE: adicione docs/prints/01-pipeline-build-test.png e rode docs/gerar-pdf.py novamente.

Pipeline: deploy em Staging (resumo do job com smoke test)

PRINT PENDENTE: adicione docs/prints/02-deploy-staging.png e rode docs/gerar-pdf.py novamente.

Pipeline: deploy em Producao (resumo do job com smoke test)

PRINT PENDENTE: adicione docs/prints/03-deploy-production.png e rode docs/gerar-pdf.py novamente.

5. Staging e Produção funcionando

Staging funcionando (porta 8081): health e login

PRINT PENDENTE: adicione docs/prints/04-staging-funcionando.png e rode docs/gerar-pdf.py novamente.

Producao funcionando (porta 8080): health e login

PRINT PENDENTE: adicione docs/prints/05-producao-funcionando.png e rode docs/gerar-pdf.py novamente.

Swagger UI com os endpoints

PRINT PENDENTE: adicione docs/prints/06-swagger-ui.png e rode docs/gerar-pdf.py novamente.

6. Desafios encontrados e como foram resolvidos

Desafio	Problema	Solução
Dependência circular no Spring Security	SecurityConfig, JwtAuthFilter e UserDetailsService formavam um ciclo.	UserDetailsServiceImpl virou @Service independente.
Testes sem Oracle no CI	O Oracle não está disponível nos runners para testes unitários.	Perfil test com H2 em memória e Flyway desabilitado; Mockito nos serviços.
Hashes bcrypt do seed	Os hashes não correspondiam às senhas documentadas.	Hashes regerados e verificados com checkpw antes de entrar no SQL.
Loop infinito na serialização JSON	Relações @ManyToOne causavam StackOverflowError.	@JsonIgnoreProperties nos campos de relação.
JWT malformado retornando 500	JwtException no filtro não era tratada.	try/catch no filtro; a cadeia segue sem autenticar e o entry point devolve 401 JSON.
Deploy apenas simulado	Os jobs de deploy só faziam echo de 'UP'.	Deploy real efêmero com Docker Compose no runner + smoke test automatizado.
Staging e produção colidindo	Mesmo projeto Compose, container_name fixo e portas repetidas faziam o staging recriar a produção.	Um compose parametrizado, projetos (-p), redes e volumes por ambiente, env/*.env.
Testes frágeis encontrados na revisão	Um stub nunca usado reprovaria o teste (strict stubs do Mockito) e o teste web mockava o filtro JWT e o ObjectMapper.	Stub removido e AuthControllerTest reescrito com filtros desligados e ObjectMapper real.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	[x]
Dockerfile funcional	[x]
docker-compose.yml (ou Kubernetes)	[x]
Pipeline com etapas de build, teste e deploy	[x]
README.md com instruções e prints	[ ]
Documentação técnica com evidências (PDF)	[ ]
Deploy realizado nos ambientes staging e produção	[ ]

Itens com [ ] dependem dos prints do primeiro run do pipeline (docs/prints/). Rode docs/gerar-pdf.py depois de adicioná-los.